Plataforma de reconocimiento de dígitos manuscritos

Búsqueda en grafos (BFS, DFS, A*) y algoritmo genético

Actividad: Plataforma que resuelve un problema
Asignatura: Fundamentos de Inteligencia Artificial

Dato	Información
Estudiante	Julissa Marín Flor
Grupo / Docente	171 - Javier García Lara
Fecha	30 de octubre de 2026

Resumen. Se diseña un sistema que recibe una imagen de 28x28 píxeles en escala de grises y decide si contiene un dígito del 0 al 9 o si debe clasificarse como *null*. El problema se formula como búsqueda en un grafo de 11 nodos (Unknown y los dígitos). Se comparan búsquedas ciegas (BFS, DFS) con una heurística (A*), y un algoritmo genético optimiza los pesos de clasificación. Todo se implementa en una página HTML funcional, sin servidor, publicable en GitHub Pages.

Sección 1. Formulación del problema como búsqueda

Entrada: imagen de 28x28 píxeles en escala de grises. Salida: una clase del conjunto {0,1,...,9,null}. La imagen se preprocesa (binarización por umbral, recorte al cuadro delimitador y reescalado a 20x20) antes de extraer rasgos.

Elemento	Definición formal
a) Estado inicial	Nodo **Unknown**: imagen 28x28 sin interpretar, con vector de rasgos vacío y costo acumulado g = 0.
b) Estados	Interpretaciones parciales: tripla (nodo actual, rasgos ya extraídos, costo g). Rasgos: número de huecos, endpoints, simetría horizontal, simetría vertical y densidad de tinta en una cuadrícula de 3x3.
c) Acciones	(1) Extraer un rasgo: contar huecos por inundación del fondo, contar endpoints sobre el esqueleto (adelgazamiento de Zhang-Suen), medir simetrías, calcular densidades. (2) Moverse por una arista del grafo hacia un dígito candidato y compararlo con su prototipo.
d) Prueba de objetivo	Se llega a un nodo-dígito n con f(n) = g(n) + h(n) <= theta (umbral de aceptación). Si el mejor f es mayor que theta, o la imagen casi no tiene tinta, el resultado es **null**.
e) Función de costo	g(n) = 1 + ws·5·(|dif. simetría H| + |dif. simetría V|) + wd·(5/3)·suma de |dif. densidad por celda|. Mide la diferencia geométrica con el prototipo de n: menor costo = mejor coincidencia.
f) Justificación	Hay estados, acciones, prueba de objetivo y costo; la solución es un camino Unknown -> dígito en un grafo, por eso es un problema de búsqueda. BFS y DFS son **búsquedas ciegas**: no usan información de qué tan cerca está cada dígito. A* es **búsqueda heurística**: usa h(n) calculada con rasgos para expandir primero el nodo más prometedor.

Sección 2. Grafo representativo

El grafo tiene 11 nodos: el nodo de inicio **Unknown** (amarillo, centro) y los nodos objetivo **0 a 9** (azul). Las líneas punteadas Unknown-dígito tienen peso **dinámico** entre 1 y 10, calculado para cada imagen. Las aristas continuas entre dígitos son fijas y su peso (número en rojo) expresa la **disimilitud**: 1 = muy parecidos, 10 = muy distintos.

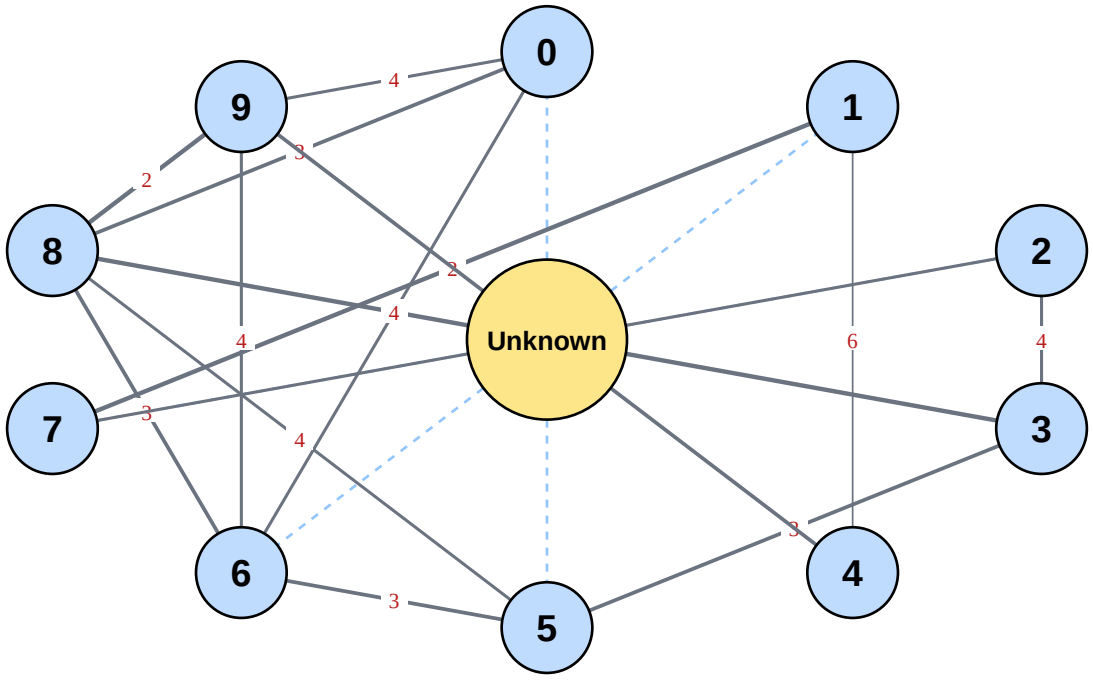

Figura 1. Grafo de reconocimiento (generado por software).

Arista	Peso	Rasgos compartidos (justificación)
1 - 7	2	Trazo vertical o diagonal, 0 huecos, 2 endpoints
3 - 8	2	Curvas del lado derecho, simetría vertical
8 - 9	2	Hueco superior y trazo curvo
3 - 5	3	Curva superior e inferior, 2 endpoints
5 - 6	3	Barra superior izquierda y curva inferior
6 - 8	3	Hueco en la parte inferior
0 - 8	3	Contorno cerrado y huecos
4 - 9	3	Hueco superior y trazo vertical derecho
0-6, 0-9, 6-9	4	Un hueco y densidad concentrada en los bordes
2-7, 2-3, 5-8	4	Trazo superior horizontal y curvas
1 - 4	6	Solo comparten el trazo vertical

Nodo de inicio: Unknown. **Nodos objetivo:** los dígitos 0 a 9. Un nodo objetivo se acepta solo si su costo total no supera theta; de lo contrario la respuesta es null.

Sección 3. Algoritmos

a) Búsqueda ciega: BFS y DFS

Ambos parten de Unknown y recorren el grafo sin información adicional. **BFS** usa una cola: expande Unknown y visita por niveles los 10 dígitos en orden (0, 1, ..., 9). **DFS** usa una pila: desde un dígito baja por sus vecinos (por ejemplo 0 -> 6 -> 5 -> 3 ...) antes de retroceder. En ambos casos se evalúa el costo f de cada nodo visitado y, al agotar el grafo, se elige el de menor costo. Siempre expanden los 11 nodos, porque ninguno sabe cuál es más prometedor.

b) Búsqueda heurística: A*

A* ordena una cola de prioridad por f(n) = g(n) + h(n), donde g(n) es el costo geométrico (Sección 1) y la heurística usa solo rasgos topológicos:

```
h(n) = wh · |huecos_imagen - huecos_n| + we · |endpoints_imagen - endpoints_n|
```

Ejemplo: una imagen con 2 huecos y 0 endpoints tiene h bajo respecto al 8 (2 huecos, 0 endpoints) y alto respecto al 1 (0 huecos, 2 endpoints), por lo que A* explora primero al 8.

Admisibilidad. Una heurística es admisible si nunca sobreestima el costo real restante h*(n). Aquí h(n) contiene únicamente la discrepancia topológica, que es una parte del costo necesario para confirmar el nodo n; el resto del costo (la parte geométrica) es siempre mayor o igual a 0. Por tanto h(n) <= h*(n) y A* no descarta la mejor solución. Con A*, tras expandir Unknown se extrae el nodo con menor f: si f <= theta, es la respuesta; si no, ningún otro nodo puede cumplirlo y se responde null. Expande solo 2 nodos frente a 11 de BFS/DFS.

c) Algoritmo genético (AG)

Componente	Diseño
Cromosoma	Vector real W = [wh, we, ws, wd, theta]. Rangos: wh en [0,6], we en [0,4], ws en [0,4], wd en [0,6], theta en [3,30].
Población	40 individuos: 1 con los pesos iniciales, 39 aleatorios (semilla fija para reproducibilidad).
Fitness	Porcentaje de aciertos al clasificar un conjunto de validación generado (100 dígitos + 20 muestras null). Los 25 dígitos de prueba no se usan para entrenar.
Selección	Torneo de tamaño 3.
Cruce	Aritmético: hijo = alfa·padre1 + (1 - alfa)·padre2, con alfa aleatorio en [0,1].
Mutación	Uniforme pequeña, probabilidad 0.25 por gen, de ±10% del rango; los valores se recortan al rango.
Elitismo	Los 2 mejores individuos pasan intactos a la siguiente generación.
Criterio de parada	Fitness >= 99.5%, o 8 generaciones sin mejora, o 40 generaciones.

d) Flujo completo

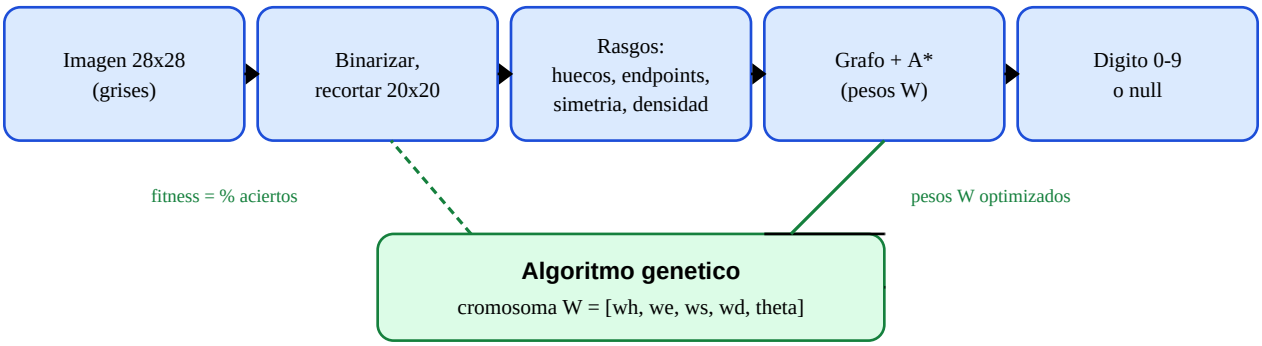

Figura 2. Flujo: red de rasgos + búsqueda + aprendizaje.

Primero se extraen los rasgos de la imagen. Luego el grafo, con los pesos W, evalúa cada dígito y A* elige el nodo de menor f. El AG repite este proceso sobre muchos conjuntos de pesos W, conserva los que más aciertos logran, los recombina y los muta. Así, los pesos pasan de valores fijados a mano a valores ajustados a los datos, y la clasificación mejora. Finalmente, la W resultante se evalúa sobre los 25 dígitos de prueba.

Sección 4. Implementación: página HTML funcional

Se entrega un único archivo **index.html** (HTML + CSS + JavaScript, sin librerías externas) publicable en GitHub Pages. Aunque la actividad menciona código Python, la lógica corre en el navegador para que la página funcione sin servidor.

Requisito de la actividad	Cómo se cumple
Cargar una imagen 28x28	Botón de selección de archivo (la imagen se reduce a 28x28 en grises) y recuadro para dibujar con el mouse.
Extraer rasgos	Huecos (inundación del fondo), endpoints (esqueleto Zhang-Suen), simetría H y V, densidad en 3x3 regiones. Se muestran en el panel de rasgos.
Clasificar 0-9 + null	Grafo de búsqueda con selector BFS / DFS / A*. Si el mejor costo supera theta o no hay tinta suficiente, devuelve null.
Animar el recorrido	Grafo SVG animado: nodo actual (naranja), frontera (amarillo), visitados (azul) y camino final (verde); Unknown en rojo si es null.
Generar y evaluar 25 dígitos	Se generan 25 dígitos (0-9 repetidos, con distintas fuentes, grosor, rotación, inclinación y desplazamiento). El botón “Evaluar los 25” marca aciertos y errores y calcula el porcentaje.
Aprendizaje	Botón “Ejecutar AG”: muestra el mejor y el promedio por generación y los pesos finales.

Pasos para publicarla en GitHub Pages

- Crear un repositorio público en GitHub.
- Subir el archivo **index.html** a la raíz del repositorio.
- En Settings > Pages elegir “Deploy from a branch”, rama main y carpeta / (root).
- Abrir el enlace https://usuario.github.io/repositorio/ al cabo de uno o dos minutos.

Resultados de la evaluación

Completa esta tabla después de abrir tu página publicada y pulsar “Evaluar los 25” antes y después de “Ejecutar AG”. No se incluyen cifras inventadas: los valores deben salir de tu ejecución real.

Momento	Aciertos (de 25)	Porcentaje
Pesos iniciales		
Después del algoritmo genético		

Limitaciones

- Los prototipos se obtienen de fuentes tipográficas del navegador, no de escritura manual real; con fotos de manuscritos la precisión baja.
- Rasgos simples provocan confusiones esperables (1 y 7, 3 y 5, 4 y 9). Un trazo mal cerrado cambia el conteo de huecos.
- Sombras, renglones o varios números en la imagen alteran el recorte y el resultado.

Conclusiones

- Formular el reconocimiento como búsqueda permite explicar paso a paso cómo se decide una clase.
- A* con heurística admisible llega al mismo resultado que BFS/DFS expandiendo muchos menos nodos (2 frente a 11).
- El algoritmo genético automatiza el ajuste de los pesos y del umbral de rechazo null, mejorando la clasificación sin ajustar a mano.
- Un enfoque con rasgos diseñados a mano es transparente y didáctico, pero menos preciso que una red neuronal entrenada con datos reales.